

AI Native 研发体系建设与落地手册

雷鹏 数据智谷 2026年8月20日 18:10 湖南

组织级实施指南 · 2026

AI Native研发体系建设与落地手册

让每个研发团队都能用同一套组织协议，把人的经验转化为可执行工作流，把 Agent 的产出转化为可验证交付，把每次交付转化为可复用的组织智能资产。

适用：产品 / 架构 / 研发 / 测试 / 平台范围：需求 → 交付 → 组织记忆形态：单文件 · 离线 · 可打印

建设原则：先把研发方法变成机器可读、机器可执行、机器可验证的组织协议，再建设 Agent 平台。

HOW TO USE

这份手册怎么用

它不是一份概念白皮书，而是一套团队可以照着启动、运行和验收的最低完整方法。公司统一底座，团队按业务风险裁剪。

1

公司级

制定宪法、资产标准、权限、评测、平台能力和统一门禁，维护组织级 Registry。

2

领域级

维护统一语言、限界上下文、领域规则、契约与高价值场景评测集。

3

团队级

选择试点工作流，配置 Agent 与 Skill，执行项目交付并回收反馈和数据。

最小成功定义：一个真实团队能在 30 天内，使用同一套 Spec、工作流和质量门禁，完成至少 3 个真实需求；交付物可追溯、结果可复现、失败可沉淀。

三条使用约束

不允许从聊天直接跳到代码：

至少要有目标、范围、验收标准和影响分析。

不允许让 Agent 自证正确：

必须由确定性工具、独立 Review 或人工门禁验证。

不允许知识只存在对话中：

稳定事实回到权威文档，执行方法回到 Skill，失败案例回到评测集。

01 · NORTH STAR

目标、边界与建设原则

AI Native 不是“给每个人一个聊天机器人”，而是重构组织的任务表达、知识供给、工作执行、质量证明与经验沉淀方式。

人的经验 → 结构化规格 → 标准工作流 → Agent 执行 → 证据化评测 → 知识沉淀 → 组织能力持续升级

建设目标

交付可预测

需求、设计、任务和代码之间可以追溯；相同输入、规则 and 工具得到可比较结果。

知识可复用

个人经验不再依赖口口相传，而是以 Spec、ADR、Skill、Workflow 和案例存在。

协作可扩展

人负责目标、取舍与责任，Agent 负责搜索、生成、执行与验证，边界清晰。

风险可治理

权限、数据、模型、工具调用、交付结果和成本均能审计与回放。

八项原则

原则	解释	落地检查
意图优先	先明确 Why / What / Acceptance，再讨论 How。	需求是否有范围、非目标和约束标准？
权威源唯一	同一事实只允许一个 Source of Truth。	字段、规则、状态是否在多处复定义？
确定性优先	能由程序完成的校验，不交给模型自由判断。	Schema、编译、测试、策略否自动执行？
证据优先	Agent 的结论必须引用代码、文档、Trace 或测试结果。	交付结论能否被第三方复核？
最小上下文	按任务路由权威信息，避免上下文污染。	是否加载了与任务无关的大量料？
人机责任分离	模型可以建议，责任人决定高风险事项。	审批点和责任人是否明确？
默认可回滚	变更小批量、可追踪、可撤销。	高风险动作是否有快照和恢复案？
失败即资产	失败模式进入守卫、Skill 或评测集。	同类问题会不会在其他团队重演？

非目标：追求无人研发、用生成代码量衡量效能、让多 Agent 自由聊天、建设脱离业务场景的“万能平台”，都不属于本体系的成功目标。

02 · OPERATING MODEL

AI Native 研发操作系统总体架构

体系由治理、意图、执行、上下文、知识、评测和平台七层组成。Agent 位于其中，不是体系本身。

治理层组织宪法、风险分级、权限策略、责任模型、资产标准、审计与成本治理

意图层战略、PRD、领域模型、Spec、Contract、ADR、Acceptance Criteria

执行层Agent、Skill、Workflow、Human Gate、CI/CD、环境与工具调用

上下文层任务路由、检索、代码图谱、权限过滤、压缩、引用、上下文装配

知识记忆层权威知识、情景记忆、经验记忆、操作记忆、组织案例与时间知识图谱

评测观测层质量、合规、效率、成本、漂移、Trace、回放、基准集与线上反馈

平台集成层Git、IDE、模型网关、MCP/工具、制品库、开发者门户、身份与密钥系统

核心对象模型

Agent

角色 + 能力 + 工具 + 策略。是执行主体，不是知识仓库。

Skill

可复用的专业作业方法，定义输入、步骤、产物、验收与失败处理。

Workflow

把任务、Skill、工具、条件、审批与状态串成可恢复的过程。

Context Pack

针对当前任务装配的最小可信上下文，包含来源和版本。

Artifact

Spec、设计、代码、测试、ADR、Trace、报告等可版本化产物。

Evaluation

对过程和结果的可重复评测，包含用例、判据、评分和证据。

03 · GOVERNANCE

组织、责任与治理机制

公司建设统一底座，但不建立一个包办所有研发的中央 Agent 团队。治理标准集中，领域知识和交付责任下沉。

三层组织模型

层级	核心责任	必须维护的资产	关键角色
公司 AI Native 委员会	战略、风险、标准、预算和跨团队仲裁	组织宪法、风险模型、北极星指标	CTO、研发效能、全、法务
AI 研发效能平台组	平台、Registry、模板、评测、可观测与赋能	Agent/Skill/Workflow 目录、模型网关、基准集	平台工程、架构、evEx、AI 工程
领域/产品研发团队	领域知识、真实交付、场景评测和结果负责	领域语言、Spec、契约、案例、团队工作流	产品、领域专家、ch Lead、研发、试

RACI 最小责任表

活动	Responsible	Accountable	Consulted	Inform
组织规则与风险等级	平台组	AI Native 委员会	安全/法务/架构	所有研发团队
领域模型与统一语言	领域团队	领域负责人	产品/架构	相关上下游
Spec 与验收标准	产品 + Tech Lead	产品负责人	研发/测试/领域专家	交付相关方

活动	Responsible	Accountable	Consulted	Inform
Agent/Skill 发布	资产作者	资产 Owner	平台/安全/使用团队	Registr 用户
代码与生产变更	研发团队	代码/服务 Owner	Agent、Reviewer、运维	业务相关方
评测集与失败案例	测试/领域团队	质量负责人	平台/研发	资产 Owner

风险分级与人工门禁

L0 自动

只读与低风险生成

查询公开知识、生成草稿、局部格式化、无副作用分析。可自动执行，保留 Trace。

L1 自动 + Review

可逆工程变更

测试、文档、小范围代码修改。允许 Agent 执行，合并前由人或独立门禁 Review。

L2 事前审批

跨模块与敏感变更

接口、数据模型、依赖、权限、共享配置。执行前必须由责任人确认影响和回滚方案。

L3 强制人工

不可逆与生产高风险

生产删除、资金、身份权限、密钥、合规决策。Agent 只能准备方案和证据，不能自行批准。

04 · DOMAIN + SPEC

以 DDD 统一语义，以 SDD 固化意图

DDD 回答“业务世界如何划分和表达”，SDD 回答“本次变化要改变什么以及如何证明”。两者共同成为 Agent 不会随意解释的事实基础。

业务目标价值、用户、指标

领域发现事件、规则、术语

上下文划分边界、Owner、关系

变更规格需求、场景、Delta

实现验收代码、测试、回写

领域资产最小集合

统一语言表：

canonical name、展示名、定义、示例、禁用同义词、Owner。

限界上下文：

职责、核心模型、上游/下游、集成契约、团队边界。

业务不变量：

任何实现都不能破坏的约束和条件。

状态机与领域事件：

状态、触发条件、守卫、事件及其语义。

场景案例：

正常、边界、异常和反例，直接转成验收与评测数据。

变更规格模型

Current Truth（当前生效规格） + Change Proposal（为什么变） +
Spec Delta（增加 / 修改 / 删除什么） + Design（如何实现与取舍） +
Tasks（怎么执行） + Evidence（如何证明） = New Truth（新的生效规格）

推荐模式：新项目可以采用 Spec → Plan → Tasks → Implement；存量项目优先采用“当前规格 + 变更 Delta + 完成后归档”的方式，避免每次重写全量文档。

规格质量门禁

完整性

目标、范围、非目标、角色、主流程、异常、验收、依赖和风险是否齐全。

可测试性

每项需求能否转化为 Given / When / Then 或明确的可执行判据。

一致性

术语、接口、状态、字段是否与领域模型和权威契约一致。

可追溯

需求 → 设计 → 任务 → 代码 → 测试 → 发布证据是否有关联标识。

05 · INTELLIGENT ASSETS

组织智能资产目录与生命周期

Prompt 只是临时表达；真正值得治理的是能跨人、跨项目、跨模型复用并能独立评测的资产。

资产	回答的问题	必备元数据	发布门槛
Policy	什么必须、允许和禁止？	范围、等级、Owner、生效日期	冲突检查、安全审
Agent	谁来完成一类职责？	角色、模型、工具、权限、退出条件	场景评测、红队测
Skill	专业任务应该怎样完成？	输入、步骤、输出、验证、失败策略	盲测可执行、产物规
Workflow	多步骤任务怎样流转？	状态、节点、条件、审批、补偿	恢复测试、异常路测试

资产	回答的问题	必备元数据	发布门槛
Template	交付物怎样标准化?	用途、Schema、示例、版本	Lint、Schema 校
Evaluation	怎样证明能力有效?	数据、判据、基线、评分器、版本	可复现、无污染、任人确认
Connector/Tool	Agent 可以调用什么?	权限、输入输出、副作用、审计	最小权限、超时与等验证

统一 Registry

ai-native-registry/ └─ policies/

组织和领域规则 └─ agents/ # 角色定义与配置 └─ skills/
专业能力包 └─ workflows/ # 可执行流程定义 └─ templates/
Spec / ADR / Review 等模板 └─ evaluations/ # 离线基准与
真实案例 └─ connectors/ # 工具、MCP、API 描述 └─
catalog.yaml # Owner、版本、依赖、评级、状态

资产状态机

Draft作者自测

Pilot限定团队试用

Certified通过评测与治理

Deprecated迁移窗口

Retired停止发现和使用

资产发布不能只看“效果不错”：必须同时满足能力评测、权限审查、成本上限、可观测、回退策略、Owner 和支持范围。

06 · AGENT COLLABORATION

Agent 协作体系

从单 Agent + 确定性工具开始。只有当任务能够真正并行、需要独立专业判断或隔离权限时，才引入多 Agent。

标准研发 Agent

Discovery Agent

澄清目标、识别领域、查找权威源、生成未知项与影响初判。

Spec Agent

把意图转成需求、场景、非目标、边界和可测试验收标准。

Architecture Agent

分析约束、方案、依赖、风险和 ADR，不代替架构责任人决策。

Implementation Agent

按已批准任务修改代码，运行最窄验证，保存证据和差异。

Test Agent

从验收标准生成独立测试，覆盖正常、边界、异常和回归。

Review Agent

从正确性、架构、安全、可维护性、性能和漂移维度审查。

何时使用多 Agent

判断	单 Agent	多 Agent
任务关系	强串行、上下文高度共享	存在可独立完成的并行子任务
专业判断	单一领域能力足够	需要安全、架构、测试等独立意见
权限	工具权限相同	必须隔离读写或生产权限

判断	单 Agent	多 Agent
质量	工具能确定性验证	需要独立 Reviewer 避免自证
代价	低沟通和上下文复制成本	并行收益明显大于编排成本

Agent 合同

name: implementation-agent mission: 在批准的 Spec 与任务边界内完成最小代码
变更 inputs: [spec_ref, task_ref, context_pack, allowed_paths] outputs:
[change_set, test_evidence, risk_notes, doc_drift] tools: [code_read,
code_write, test_runner] permissions: workspace_write; production_none
must: - 修改前完成影响分析 - 保留用户已有变更 - 运行与风险匹配的最窄测试
must_not: - 擅自扩大需求范围 - 绕过测试或质量门禁 stop_when: - Spec 冲突或
权威源不唯一 - 影响范围达到高风险 - 需要新的权限或不可逆操作

07 · DELIVERY WORKFLOW

标准 AI Native 研发流程

主流程设置八个阶段。每个阶段都必须定义输入、动作、产物、自动门禁、人工责任和退出条件。

1 受理目标与分级

2 发现领域与影响

3 规格需求与验收

4 设计方案与任务

5 实现代码与局测

6 验证测试与 Review

7 交付审批与发布

8 学习回写与评测

阶段 1：任务受理与风险分级

输入：业务诉求、缺陷、技术任务或事故。产物：任务卡、目标、范围、初始风险等级和责任人。

识别业务价值、受影响用户和成功指标。

区分新需求、Bug、重构、实验和高风险操作。

没有 Owner、目标或验收方向的任务不得进入下一阶段。

阶段 2：领域发现与影响分析

阶段 3：Spec 与验收标准

阶段 4：设计、ADR 与任务拆分

阶段 5：实现与局部验证

阶段 6：独立验证与 Review

阶段 7：合并、发布与运行验证

阶段 8：文档回写与组织学习

08 · CONTEXT ENGINEERING

上下文工程：给 Agent 正确的信息，而不是更多信息

上下文是一条受治理的供应链：识别任务、路由权威源、过滤权限、按需压缩、保留引用，并通过反馈持续修正。

任务分类意图 / 风险 / 领域

源定位权威源 / Owner

检索过滤相关性 / 权限

上下文装配压缩 / 引用 / 预算

反馈回写命中率 / 缺失 / 污染

五层上下文

层级	内容	加载策略	示例
L0 组织	稳定红线与通用责任	始终加载，极短	安全、Git、数据、目原则
L1 项目/模块	架构边界与局部规范	进入作用域时加载	模块 AGENTS、编砒范
L2 当前任务	Spec、设计、任务、验收	任务全程保留	change proposal、sks
L3 动态证据	代码符号、调用链、测试、Trace	按步骤检索	影响分析、失败日志近
L4 历史知识	案例、事故、旧决策、外部资料	按需检索和引用	相似故障、旧 ADR

Context Pack 标准

任务摘要、目标、范围、非目标和风险等级。

命中的组织/模块规则，标明优先级和适用范围。

相关 Spec、契约、ADR、代码符号与测试。

已知事实、待确认假设、冲突和未知项分开呈现。

每条材料包含来源、版本/提交、Owner、更新时间和可信等级。

明确 Token/成本预算、过期条件和不得外泄的敏感等级。

上下文质量指标

Precision注入内容中真正相关的比例

Recall完成任务所需关键信息的覆盖率

Freshness信息版本与当前事实的一致程度

Authority权威源和 Owner 可确认比例

Traceability结论可回链到原始证据的比例

Efficiency每个成功任务的上下文成本

09 · KNOWLEDGE & MEMORY

知识与记忆体系

知识是经确认的组织事实；记忆是基于经历形成、可能过期或被纠正的信息。两者必须分层治理，不能把对话历史直接升级成公司知识。

类型	内容	推荐载体	写入规则
权威知识	PRD、Spec、Contract、Policy	Git、文档门户、契约仓库	PR 审批，Owner 责
领域知识	术语、模型、规则、事件	领域目录、知识图谱	领域负责人确认
情景记忆	一次任务发生了什么	Trace、任务记录、日志	自动记录，设保留期
经验记忆	什么有效、为什么失败	案例库、Playbook、复盘	证据化提炼后发布
操作记忆	某类任务怎样执行	Skill、Workflow、脚本	通过盲测和版本注
偏好记忆	团队/个人非强制偏好	有作用域的 Memory Store	可查看、可删除、过期

记忆写入管道

捕获事件与证据

分类事实 / 经验 / 偏好

校验来源 / 冲突 / 隐私

固化版本 / Owner / TTL

召回范围 / 排序 / 引用

必须支持的记忆治理

作用域隔离：个人、团队、项目、领域、公司不能默认互通。

时间有效性：记录 valid-time、observed-time、过期时间和被替代关系。

冲突处理：新记忆不得无提示覆盖权威知识。

隐私与删除：敏感信息分类、最小保留、可查询、可纠正、可删除。

召回解释：Agent 必须说明使用了哪条记忆以及它如何影响决策。

落地顺序：先用 Git + Markdown + 元数据 + 混合检索建立权威知识；出现大量跨项目关系、历史有效期和复杂召回需求后，再引入时间知识图谱或专用 Memory Layer。

10 · PLATFORM

平台与工具链参考架构

平台的职责是提供受治理的公共能力，不是把研发过程全部锁进单一产品。资产和规格应保持开放、可迁移、可版本化。

体验入口IDE / CLI / Chat / Dev Portal / Issue & PR / API

workflow编排任务状态机、暂停恢复、并行、人工审批、补偿、超时

Agent Runtime角色、会话、工具循环、沙箱、并发、配额、隔离

Context Service搜索、RAG、代码图谱、权限过滤、装配、引用和缓存

Model Gateway模型路由、密钥、数据策略、限流、缓存、成本、故障转移

Tool GatewayMCP/API/CLI Registry、身份委托、参数校验、审计与副作用控制

ObservabilityTrace、Prompt/Context 版本、调用、Token、延迟、结果、评测与告警

Build / Buy / Adopt 判断

能力	建议	原因
模型本身	Buy / 多模型接入	不把组织能力绑定到单模型，网关统一治理。
Spec 与资产格式	Adopt + Customize	基于开源标准定制，确保内容留在 Git。
领域知识	Build	属于公司的核心语义资产，无法外购。
Agent Runtime	先 Adopt	先验证场景，避免过早自建通用编排引擎
权限与审计	Integrate / Build	必须接入现有 IAM、密钥和审计系统。
评测集	Build	真实业务任务与失败案例才有组织区分度

平台非功能要求

会话和工作流可暂停、恢复、重放和取消。

每次调用可定位模型、Prompt、Skill、Context、Tool 和输出版本。

工具调用具备超时、重试、幂等、限流、熔断和补偿语义。

支持模型与工具替换，不把业务资产锁在供应商私有格式中。

平台故障时研发仍可使用 Git 中的 Spec、模板和脚本继续工作。

11 · SECURITY

安全、权限与合规

Agent 不是共享的超级账号。它代表某个人或/服务，在明确作用域内获得临时、最小、可审计的权限。

六条安全基线

身份可追溯

区分用户、Agent、Workflow 与工具身份，记录委托链。

最小权限

按任务、路径、资源和时效授予权限，默认只读。

敏感数据隔离

模型和工具调用前进行分类、脱敏、地域与供应商策略检查。

副作用分级

读、写、删除、发布、付款等操作使用不同审批和凭据。

完整审计

保留上下文来源、工具参数、响应摘要、审批人和最终结果。

供应链治理

模型、插件、MCP、依赖、Prompt 和 Skill 均纳入版本与风险审查。

必须阻断的情况

把密钥、生产数据或个人敏感信息直接放入未批准模型上下文。

Agent 自行扩大工具权限、绕过审批或使用不明来源的 MCP Server。

生产删除、权限授予、资金或合规决定由 Agent 自动批准。

工具调用不可审计，或无法识别发起人、Agent、参数和结果。

外部内容中的 Prompt Injection 能直接驱动高风险工具。

Agent 生成的依赖、代码或制品未经常规供应链扫描。

强制规则：任何无法解释权限来源、无法回放执行证据、无法安全撤销的自动化，不得进入生产级 Workflow。

12 · EVALUATION

评测、可观测与效能指标

“感觉更快”无法支撑组织投入。建立离线基准、上线门禁、在线观测和业务结果四层评测，并同时衡量质量、速度、成本与风险。

四层评测

E1

组件评测

检索准确率、工具调用正确率、结构化输出合规率、Skill 步骤完成率。

E2

任务评测

真实需求、Bug、Review 和迁移任务的成功率、返工率与证据完整性。

E3

流程评测

从受理到交付的端到端周期、人工等待、门禁命中、失败恢复。

E4

业务评测

缺陷逃逸、交付吞吐、业务指标、员工体验、合规事件和总体成本。

指标体系

维度	推荐指标	避免误用
质量	首次通过率、缺陷逃逸率、回滚率、Spec 漂移率	不能只统计生成代码是否编译
流动效率	Lead Time、Cycle Time、人工等待时间、批次大小	不要用 Agent 调用次数代效率

维度	推荐指标	避免误用
自动化	无人工修正完成率、门禁自动化率、恢复成功率	自动化率越高不一定风险越低
资产复用	Skill 复用率、评测覆盖、资产活跃度、重复 Prompt 降幅	资产数量不是价值
上下文	召回覆盖、引用正确率、过期率、Token/成功任务	上下文越长不代表越好
成本	每个成功任务成本、返工成本、人工节省、平台 TCO	不要只看模型 Token 单
风险	越权阻断、敏感信息泄露、错误自动批准、审计完整率	零告警可能意味着没有检测能力

评测集建设

优先选择过去 3 ~ 6 个月真实发生的高频任务与高损失缺陷。

每个用例保存输入、所需上下文、阶段预期、最终判据和禁止行为。

区分黄金集、回归集、红队集和探索集，防止训练/调优污染。

模型、Skill、规则、工具或上下文策略变化后自动重跑。

评测失败必须能定位到上下文、推理、工具、流程或判据中的具体一层。

13 · 90-DAY ROADMAP

90 天落地路线

用一个真实团队和一条真实价值流证明闭环，再逐步复制。第一阶段不要建设大平台，也不要追求覆盖全部研发场景。

第 0 ~ 15 天：基线与选择试点

目标：选准问题并建立可比较基线。

选择 1 个业务清晰、反馈快、风险中等的团队。

抽取 20 ~ 50 个历史真实任务，记录周期、返工、缺陷和成本。

盘点现有 PRD、文档、规则、工具、知识库和权限。

输出试点章程、成功指标、Owner 与停止条件。

第 16 ~ 30 天：建立最小协议

目标：让需求和交付物机器可读。

发布组织宪法 v0.1、风险分级和人工门禁。

确定 Spec、Design、ADR、Task、Review 模板。

建立领域术语表和最小 Context Pack。

实现 3 ~ 5 个高频 Skill：发现、Spec、影响分析、测试、Review。

第 31 ~ 60 天：跑通交付闭环

目标：完成不少于 3 个真实需求。

把受理 → Spec → 实现 → 验证 → 回写串成 Workflow。

接入 Git/CI，建立 Schema、测试、Review 和漂移门禁。

记录完整 Trace、人工修改、失败点、Token 与周期。

每周评测并更新 Skill，不在执行中临时堆叠 Prompt。

第 61 ~ 90 天：产品化与复制

目标：形成可被第二个团队自助使用的版本。

建设最小 Registry 和版本/Owner/状态治理。

发布团队启动包、培训、办公时间与支持 SLA。

用盲测验证另一个团队能否不依赖原作者完成启动。

形成季度路线：平台缺口、资产清单、规模化预算和风险。

首批推荐场景

代码影响分析

输入清晰、证据可查、价值直接，是 Context Engineering 的好试点。

Spec 与验收生成

连接产品和研发，能快速暴露术语、权威源和需求质量问题。

代码 Review

适合建立独立验证、规则门禁和失败回归集。

测试生成与执行

结果较确定，容易衡量覆盖、通过和返工。

故障排查

能沉淀 Trace、案例和经验记忆，但需要严格控制生产权限。

文档漂移检查

连接 Spec、Contract 和 Code，构成组织学习闭环。

14 · MATURITY

成熟度模型

团队不能靠宣称“已 AI Native”升级。必须以流程、资产、质量、治理和业务结果的可验证证据晋级。

等级	特征	晋级证据
L0 工具尝试	个人聊天、代码补全，规则和知识分散	无统一要求
L1 辅助研发	有团队规则和少量模板，人工主导全部流程	规则入库、结果需 Review，基线可测

等级	特征	晋级证据
L2 标准流程	Spec 驱动, Skill/Workflow 可复用, CI 有门禁	3+ 真实交付、可追溯、评估集稳定
L3 规模协作	多团队共享资产, 统一 Registry、权限和可观测	第二团队自助接入、资产所有者/SLA
L4 数据优化	基于 Trace、评测和业务指标持续优化路由与流程	质量与周期有统计显著改善
L5 自适应组织	失败自动进入评测, 资产按证据演进, 跨团队知识复利	闭环可审计, 重大决策仍由负责

建议目标：大多数团队先稳定达到 L2；平台组帮助重点团队达到 L3。不要为了“高级”而过早建设 L4/L5 的复杂基础设施。

15 · TEAM PLAYBOOK

研发团队启动手册

以下步骤供任意团队在两周内完成最小接入。团队负责人对结果负责，平台组负责模板、工具和辅导。

Day 1 ~ 2：界定范围

指定团队 AI Native Owner、领域 Owner 和质量 Owner。

选择一个价值流和 3 个近期真实任务。

记录当前周期、返工、缺陷和人工投入基线。

确定禁止自动化的操作和数据。

Day 3 ~ 5：准备知识与规则

确认模块规则、架构边界和技术规范。

整理前 20 个核心术语、主要业务规则和接口权威源。

补齐 Spec、ADR、Review 模板以及目录约定。

删除重复、过时或互相冲突的指令。

Day 6 ~ 8：配置能力

选择批准的模型和研发入口。

启用影响分析、Spec、实现、测试和 Review Skill。

按风险配置工具白名单、可写路径和人工审批。

把最窄测试、Lint、契约校验接入自动门禁。

Day 9 ~ 10：盲跑与上线

由未参与配置的成员完成一条端到端任务。

记录缺失上下文、误解、无效步骤、人工修正和成本。

修复流程或 Skill，而不是只追加临时 Prompt。

通过上线清单后进入限定范围试运行。

团队每周节奏

频率	活动	产物
每个任务	Spec、影响分析、证据化交付、反馈	任务 Trace 与产物链接
每周	失败案例 Review、Skill/规则变更评审	新增评测与改进清单
每两周	效能和质量指标复盘	对比基线、删除无效流程
每月	资产健康和权限审计	升级、废弃、成本和风险报告
每季度	组织能力与投资组合评审	路线图和规模化决策

16 · TEMPLATES

可直接复用的模板工具箱

模板是最低信息协议，不是要求所有任务写长文。低风险任务可以简化字段，高风险任务必须补全证据、审批和回滚。

A. Change Proposal复制

变更提案：[标题] ## Why - 问题/机会： - 目标用户与价值： - 成功指标： ## Scope - 本次范围： - 非目标： - 受影响领域/系统： ## What Changes - 新增： - 修改： - 删除： ## Risk - 风险等级： L0 / L1 / L2 / L3 - 兼容、数据、权限、安全与运行风险： - 回滚/降级方向： ## Ownership - Product Owner： - Tech Owner： - Domain Owner：

B. Requirement & Acceptance复制

REQ-001 [需求名称] 作为 [角色] 我希望 [能力] 从而 [业务价值] ### 规则与约束 - MUST： - MUST NOT： ### 场景 Given [前置条件] When [动作/事件] Then [可观测结果] ### 边界与异常 - 边界： - 异常： - 降级： ### Traceability - Domain： - Design： - Task： - Test：

C. Architecture Decision Record复制

ADR-NNNN：[决策标题] - Status: Proposed / Accepted / Deprecated / Superseded - Date: - Owners: - Related Spec: ## Context 问题、约束、驱动因素和不可改变条件。 ## Options 1. 方案 A：优势 / 劣势 / 风险 2. 方案 B：优势 / 劣势 / 风险 ## Decision 选择什么，以及为什么。 ## Consequences - 正向影响： - 负向影响： - 后续动作： - 何时重新评估：

D. Skill Contract复制

name: skill-name purpose: 该 Skill 解决什么问题 triggers: 何时必须/可以使用
non_goals: 不解决什么 inputs: - name / schema / required preconditions: -
权威源、权限、环境要求 steps: - 动作、工具、产生的中间证据 outputs: - artifact
/ schema / location quality_gates: - 确定性检查与人工门禁 stop_conditions: -
冲突、越权、高风险或信息不足 failure_strategy: - 重试、降级、回滚、升级
evaluation: - 用例集、指标、通过阈值 owner: team/name version: semver

E. Workflow Definition复制

name: feature-delivery entry_criteria: - 目标、Owner、风险等级已确认 states:
intake: actor: human + discovery-agent output: task-card next:
discovery discovery: actor: discovery-agent output: context-pack,
impact-map gate: high-risk-human-approval spec: actor: spec-agent
output: proposal, requirements, acceptance gate: product-owner-approval
implement: actor: implementation-agent output: change-set, test-
evidence review: actor: review-agent + code-owner output: review-
report learn: actor: workflow output: spec-update, eval-cases timeouts:
... retry_policy: ... compensation: ... audit: enabled

F. Agent 交付报告复制

Outcome 完成了什么，是否满足目标。 ## Changes - 变更文件/系统： - 关键行为
变化： ## Evidence - 测试与结果： - 引用的 Spec / ADR / Contract： - Trace /
日志： ## Risk & Limits - 未覆盖项： - 已知风险： - 假设： ## Drift - 文档/契约
是否需要同步： ## Human Decision Needed - 需要谁决定什么；若无则写“无”。

17 · OPEN SOURCE

开源项目选型建议

选型原则是借鉴成熟模型、保持资产开放、从试点验证，不把组织方法绑定到单个框架。

能力	项目	适用场景	建议
Spec 驱动	GitHub Spec Kit	新项目、标准 Spec → Plan → Tasks → Implement	用于公司模板、和工作流参考

能力	项目	适用场景	建议
增量 Spec	OpenSpec	存量系统、以变更 Delta 演进当前事实	优先用于大型存量码库试点
Agent 研发方法	BMAD Method	完整产品/架构/研发/测试角色与阶段	借鉴方法，裁剪后应用
DDD 建模	DDD Crew Starter Process	领域发现、统一语言、边界与团队设计	作为领域工作坊核心流程
架构决策	MADR	用 Markdown 记录架构决策与后果	直接采用或轻量定制
开发者门户	Backstage + TechDocs	软件目录、Owner、模板、Docs as Code	团队规模和服务类足够后引入
工作流 Runtime	Microsoft Agent Framework	Python/.NET 生产级 Agent 与工作流	平台自建 Runtime 时评估
有状态 Workflow	LangGraph	暂停恢复、人工介入、长流程和图编排	适合复杂确定性流 + Agent 节点
通用记忆	Mem0	Agent 长期记忆、自托管 Memory Layer	在记忆边界和评测确认后试点
时间知识图谱	Graphiti	实体关系随时间变化、复杂历史召回	不要作为第一阶段依赖

推荐组合

DDD Crew
与变更 MADR

→ 统一领域语言与边界 OpenSpec / Spec Kit
→ 保存架构决策 Git + CI

→ 管理意图、规格
→ 版本、审批与确定性门

禁 Agent Skills → 封装可复用专业方法 Evaluation Suite → 用真实任务证明能力 Backstage → 规模化后的资产发现入口 Agent Runtime → 确有平台级长流程需求时建设

选型结论：没有一个开源项目能够独立提供完整的组织级 AI Native 研发体系。开源框架提供结构和运行能力，公司的领域语义、工程规则、真实评测与治理责任才是核心壁垒。

18 · ANTI-PATTERNS

常见反模式与纠正方式

反模式	后果	纠正方式
Prompt 大全	版本混乱、难以评测、无法组合	沉淀为带合同、脚本和评测的 S
多 Agent 角色扮演	沟通成本高、互相确认、错误被放大	确定性 Workflow 为主，仅在必要时分工
文档越多越好	上下文污染、重复事实、维护成本失控	权威源唯一、分层路由、Delta 新
先建大平台	脱离业务、半年后仍无真实闭环	一团队、一流程、三需求证明后平台化
AI 自己 Review 自己	共享盲区，自证偏差	独立 Reviewer + 确定性测试 + 人工责任
把聊天当记忆	事实不可控、过时、泄露和错误传播	记忆分类、验证、作用域、TTL 引用
只看速度	返工、缺陷和风险被隐藏	质量、流动、成本、风险平衡计

反模式	后果	纠正方式
默认超级权限	越权和不可逆事故	身份委托、最小权限、短时凭据 人工门禁
模型锁定	成本、供应和能力受单点制约	模型网关、开放资产格式、多梯 评测
失败只修当前点	同一根因跨团队重复发生	形成契约、守卫、不合规矩阵和 归用例

19 · ACCEPTANCE

公司级与团队级验收清单

以下条件满足后，才能宣称“已搭建可用的 AI Native 研发流程”；使用了某个模型、IDE 或多 Agent 框架不构成验收证据。

公司级最低验收

组织宪法、风险分级和人工审批矩阵已发布。

Agent/Skill/Workflow/Evaluation 有统一资产标准。

模型和工具通过网关进行身份、权限和审计治理。

存在真实任务评测集、基线和版本变更回归机制。

资产有 Owner、版本、状态、依赖、支持范围和废弃机制。

质量、效率、成本和风险进入统一仪表盘。

至少两个团队能够自助采用而不依赖原始建设者。

团队级最低验收

有明确的团队 Owner、领域术语和模块规则。

真实任务遵循 Spec → Design → Task → Evidence。

高风险变更有影响分析、人工门禁和回滚方案。

Agent 只能访问任务所需的最小工具和路径。

测试和 Review 与实现过程独立，结果可复现。

代码、契约和文档漂移在交付前被检查。

失败案例已进入 Skill、守卫或评测集。

连续完成至少 3 个真实需求并优于基线。

最终验收问题

如果明天更换模型、人员或 IDE，这套流程、知识、评测和责任机制是否仍然成立？如果答案是否定的，它还不是组织能力。

建议的下一步

任命一名公司级 AI Native 负责人，并选定一个试点团队。

用本手册模板建立试点章程、组织宪法 v0.1 和评测基线。

选择 OpenSpec 风格的变更流程，先实现 3~5 个 Skill。

在 30 天内完成第一批真实需求，用证据决定继续、调整或停止。